

- Déploiement et transfert de mes serveurs — de l'ancien HP à Proxmox « dino »

- Introduction

- Contexte professionnel

- Technologies utilisées

- Architecture du Réseau

- Tableau du plan d'adressage IP

- Tableau des spécifications

- Configuration des Équipements

- 1. Sauvegarde de l'existant avant toute migration

- 2. Sécuriser les identifiants exposés avant de migrer

- 3. Installer le socle Proxmox + IaC sur draken-admin

- 4. Corriger deux bugs de permissions Proxmox pendant le premier

```
terraform apply
```

- 5. Débloquer un

```
terraform apply
```

qui restait bloqué 13 minutes

- 6. Transférer les dépôts Gitea sans exposer de token en clair

- 7. Migrer la base PostgreSQL de N.C0re sans perte

- 8. Transférer et redéployer les services applicatifs

- 9. Reconvertir l'ancien serveur en cible de sauvegarde

- Tests et Résultats

- Difficultés Rencontrées et Solutions

- Améliorations Possibles

- Ce que ce Projet Démontre

- Références et Ressources

Déploiement et transfert de mes serveurs — de l'ancien HP à Proxmox « dino »

Migration complète de mon ancien serveur physique vers un nœud Proxmox VE, avec sauvegarde, transfert des données et redéploiement de chaque service.

Auteur : Garlens Charles

Formation : BTS SIO – Option SISR

Année : 2025-2026

Introduction

Mon ancien serveur (2 cœurs, 3,7 Go de RAM, en Wi-Fi) commençait à montrer ses limites, et je voulais une vraie plateforme de virtualisation pour progresser. J'ai donc tout migré vers un nouveau serveur sous Proxmox VE, que j'appelle « dino ». Ce document reprend comment j'ai sauvegardé l'ancien serveur, transféré les données, et redéployé chaque service sans perte — avec tous les blocages que j'ai rencontrés en chemin.

Contexte professionnel

- **Plan de continuité d'activité** : sauvegarder avant de toucher à quoi que ce soit (

```
vzdump
```

```
,
```

```
pg_dumpall
```

) est le réflexe qu'on attend en entreprise avant toute migration risquée.

- **Zéro-downtime perçu** : j'ai gardé l'ancien serveur allumé et intact pendant toute la migration, pour pouvoir revenir en arrière si besoin (rollback).
- **Séparation des responsabilités** : mon architecture finale sépare pare-feu (OPNsense), pivot d'administration (draken-admin), et services applicatifs (VM apps) — un principe qu'on retrouve dans toute infra un peu sérieuse.
- **Gestion des secrets pendant une migration** : j'ai dû régénérer plusieurs tokens (Proxmox, Gitea) exposés dans un ancien script, plutôt que de les transférer tels quels.
- **Traçabilité** : chaque étape de la migration est passée par Terraform/Ansible ou documentée, pas juste faite « à la main puis oubliée ».

Technologies utilisées

- Proxmox VE 9.2
— hyperviseur du nouveau serveur
- vzbump
— sauvegarde native Proxmox (snapshots avant modification)
- Terraform
(provider bpg) +
Ansible
— provisioning et configuration des nouvelles machines
- Docker
+
docker compose
— exécution des services applicatifs, avant et après migration
- pg_dumpall
/
pg_restore
— export/import PostgreSQL sans corruption

Architecture du Réseau

L'ancien serveur (100.121.13.37) hébergeait tout en une seule machine Docker. Le nouveau (dino, 192.168.1.50) sépare les rôles : un pare-feu OPNsense en frontal, un bastion d'administration dédié (draken-admin) qui pilote tout en Terraform/Ansible, et des machines applicatives séparées (LXC pour les services système, VM pour Docker).

Tableau du plan d'adressage IP

Équipement	Adresse IP	Rôle
Ancien HP	100.121.13.37 (Tailscale)	Serveur source, migré puis reconverti en cible de sauvegarde
dino (nœud Proxmox)	192.168.1.50	Hyperviseur
draken-admin (CT 102)	192.168.2.101	Bastion Terraform + Ansible, pivot de tous les transferts
apps (VM 120)	192.168.2.20	Docker, cible finale des services applicatifs
db (CT 113)	192.168.2.13	PostgreSQL, cible de la base migrée
git (CT 112)	192.168.2.12	Gitea, cible des dépôts migrés

Tableau des spécifications

Ressource	Valeur
Ancien HP	2 cœurs, 3,7 Go RAM, Wi-Fi
dino (nœud)	16 cœurs, 15 Gio RAM, 94 Go disque
VM apps	4 vCPU, 4 Go RAM (floating 3 Go), 40 Go disque
CT db	2 cœurs, 1,5 Go RAM, 20 Go disque

Configuration des Équipements

1. Sauvegarde de l'existant avant toute migration

Pourquoi cette étape est importante ? Je ne voulais surtout pas perdre des données réelles (dépôts Git, base N.C0re) en cas d'erreur pendant le transfert. Un dump propre est plus fiable qu'une copie à chaud d'un volume Docker en cours d'écriture.

```
# Dump propre de Postgres (mieux qu'un tar à chaud)
docker exec ncore-postgres pg_dumpall -U postgres > ~/backups/ncore-pg-$(date +%F).sql

# Tar des volumes de données
for V in gitea_data dns-auto_pihole_data nginx_data nginx_letsencrypt portainer_data; do
    docker run --rm -v $V:/data -v ~/backups:/backup alpine \
        tar czf /backup/$V.tar.gz -C /data . ; done

# Archive du code (sans node_modules/.next, ~20 Go de cache à ne pas transférer)
tar czf ~/backups/projets-$(date +%F).tar.gz \
    --exclude='node_modules' --exclude='.next' --exclude='__pycache__' \
    ~/doc-forge ~/portfolio ~/n-core-api ~/generateur-cv ~/intelligence-hub \
    ~/kb-webhook ~/knowledge-base ~/scripts

crontab -l > ~/backups/crontab.txt
```

Résultat : toutes les données critiques sauvegardées avant de toucher au HP.

2. Sécuriser les identifiants exposés avant de migrer

Pourquoi cette étape est importante ? J'ai découvert que mon script

```
migrate.sh
```

avait des tokens GitHub/Gitea en clair. Les transférer tels quels sur la nouvelle infra aurait juste déplacé le problème de sécurité au lieu de le régler.

```
# Token GitHub exposé (ghp_...) → révoqué manuellement sur GitHub, un nouveau  
# généré au besoin  
# Le remote Gitea embarquait le mot de passe en clair dans l'URL : corrigé  
# avec un token dédié  
git remote set-url origin http://dino:<TOKEN>@192.168.2.12:3000/dino/  
portfolio.git
```

Résultat : plus aucun secret en clair dans un fichier ou une URL versionnée.

3. Installer le socle Proxmox + IaC sur draken-admin

Pourquoi cette étape est importante ? Plutôt que de piloter la migration depuis mon PC perso, j'ai voulu un poste dédié (bastion) qui garde le token API et le state Terraform au même endroit, réutilisable pour toutes les futures machines.

```
[ -f ~/.ssh/id_ed25519 ] || ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519  
mkdir -p ~/draken-infra/terraform/modules/{lxc,vm,network}  
rm -f ~/draken-infra/terraform/terraform.tfvars  
# supprimé exprès, c'est le piège à secret  
cd ~/draken-infra && git init -b main  
cat > .gitignore <<'EOF'  
.terraform/  
*.tfstate*  
*.tfvars  
*.bak*  
EOF
```

Résultat : le token API vit uniquement dans

```
~/.secrets/proxmox.env
```

, jamais dans un fichier commité.

4. Corriger deux bugs de permissions Proxmox pendant le premier

terraform apply

Pourquoi cette étape est importante ? C'est le blocage le plus long de toute la migration (plusieurs heures perdues) — comprendre la séparation de privilèges Proxmox m'a évité de refaire la même erreur sur toutes les machines suivantes.

```
# Erreur au premier apply : "permission denied" malgré un token qui semblait
    avoir les droits
pveum acl modify / -token 'terraform@pve!terraform' -role PVEAdmin
pveum user permissions 'terraform@pve!terraform'
    # doit lister VM.Allocate, Datastore.Allocate...

# Erreur suivante, spécifique à la VM apps : "Sys.Modify" manquant pour
    l'import du disque cloud-init
pveum role add SysModifyOnly -privs Sys.Modify
pveum acl modify / -user terraform@pve -role SysModifyOnly
pveum acl modify / -token 'terraform@pve!terraform' -role SysModifyOnly
```

Résultat :

terraform apply

passé enfin — j'ai compris qu'un token Proxmox n'hérite **jamais** automatiquement des droits de son utilisateur parent ; ses droits effectifs sont l'intersection des deux, il faut donner le rôle aux deux séparément.

5. Débloquer un

terraform apply

qui restait bloqué 13 minutes

Pourquoi cette étape est importante ? Ça ressemblait à un plantage silencieux (aucune erreur affichée), alors que la VM tournait déjà et répondait en SSH — un cas où il fallait comprendre ce que Terraform attendait exactement, pas juste relancer la commande en boucle.

```
# Terraform restait sur "Still creating..." alors que la VM répondait déjà en
    SSH
ssh debian@192.168.2.20 "sudo apt install -y qemu-guest-agent && sudo
    systemctl enable --now qemu-guest-agent"
```

Résultat : Terraform s'est débloqué immédiatement une fois le

```
qemu-guest-agent
```

installé — il attendait ce signal pour considérer la VM comme prête, et l'image cloud Debian utilisée ne l'embarque pas par défaut.

6. Transférer les dépôts Gitea sans exposer de token en clair

Pourquoi cette étape est importante ? Ma première idée (générer un token Gitea et faire un

```
git push --mirror
```

classique) aurait matérialisé un secret complet en ligne de commande. J'ai préféré une méthode qui n'expose jamais de token, même si elle demande une étape en plus.

```
# Sur l'ancien HP : archive brute des dépôts (pas de push réseau)
docker exec gitea tar czf /tmp/dino-repos.tar.gz -C /data/git/repositories/
dino .
docker cp gitea:/tmp/dino-repos.tar.gz ~/dino-repos.tar.gz

# Transfert via draken-admin (pivot : mon PC n'a pas de clé directe vers les
LXC du lab)
scp ~/dino-repos.tar.gz dino@192.168.2.101:~/
ssh dino@192.168.2.101 "scp ~/dino-repos.tar.gz root@192.168.2.12:/tmp/"

# Sur le LXC git : extraction puis adoption (Gitea ne "voit" pas un dépôt
juste posé sur le disque)
ssh root@192.168.2.12 "tar xzf /tmp/dino-repos.tar.gz -C /var/lib/gitea/
repositories/dino/ && \
    chown -R git:git /var/lib/gitea/repositories/dino/"
curl -u dino:<mdp> -X POST http://192.168.2.12:3000/api/v1/admin/unadopted/
dino/portfolio-v2
```

Résultat : les 10 dépôts (doc-forge, portfolio-v2, n-core, generateur-cv, etc.) apparaissent sous mon compte

```
dino
```

avec leur contenu réel, sans qu'aucun token n'ait transité en clair sur le réseau.

7. Migrer la base PostgreSQL de N.C0re sans perte

Pourquoi cette étape est importante ? C'était la seule donnée « vivante » de toute la migration (pas juste du code) — je voulais une preuve concrète que rien n'avait été perdu, pas juste une confiance aveugle dans le dump.

```
# Redémarrer le postgres de l'ancien HP (arrêté, volume intact)
ssh -p 2222 gls@100.121.13.37 "cd ~/n-core-api && docker compose up -d ncore-
postgres && sleep 5"

# Dump depuis le HP → transfert local sur draken-admin → restore dans le LXC
db
ssh -p 2222 gls@100.121.13.37 "docker exec ncore-postgres pg_dumpall -U
postgres" > /tmp/ncore.sql
scp /tmp/ncore.sql root@192.168.2.13:/tmp/
ssh root@192.168.2.13 "sudo -u postgres psql -f /tmp/ncore.sql"

# Vérification : comparer les tables des deux côtés
ssh root@192.168.2.13 "sudo -u postgres psql -d ncore -c '\dt'"
# → 6 tables : body_stats, exercises, programs, users, workout_sessions,
workout_sets
```

Résultat : les 6 tables sont identiques des deux côtés (0 ligne partout, l'app n'avait pas encore de données réelles — mais la structure et le rôle sont bien migrés à l'identique).

8. Transférer et redéployer les services applicatifs

Pourquoi cette étape est importante ? Le transfert direct de mon PC vers la VM apps était refusé (clé SSH non fiée), donc j'ai dû comprendre comment router correctement les fichiers via mon bastion.

```
# Erreur rencontrée : --strip-components=3 aplatissait tous les projets ensemble
# (les chemins dans l'archive sont home/gls/<projet>/..., donc 2 niveaux avant le nom, pas 3)
tar xzf projets-*.tar.gz --strip-components=2 -C ~/

# Méthode de transfert qui fonctionne : PC → draken-admin (scp) → apps (scp depuis draken-admin)
scp ~/backups/projets-*.tar.gz dino@192.168.2.101:~/
ssh dino@192.168.2.101 "scp ~/projets-*.tar.gz debian@192.168.2.20:~/ "

# Redéploiement service par service, dans l'ordre des dépendances
cd ~/n-core-api && docker compose up -d # après avoir pointé DB_HOST vers 192.168.2.13
cd ~/doc-forge && docker compose up -d --build
docker ps && curl -I http://localhost:5000
```

Résultat : les 8 services applicatifs redéployés et vérifiés un par un (voir tableau des tests).

9. Reconvertir l'ancien serveur en cible de sauvegarde

Pourquoi cette étape est importante ? Plutôt que de laisser l'ancien serveur inutilisé, j'ai voulu lui donner un second rôle utile : cible de sauvegarde pour la nouvelle infra, avec de vraies restaurations testées.

```
sudo apt install -y proxmox-backup-server
# GUI PBS (https://100.121.13.37:8007) : créer un datastore + un token API
```

```
# Sur dino : brancher le nouveau serveur comme cible PBS + premier backup complet
pvesm add pbs pbs-hp --server 100.121.13.37 --datastore <nom_datastore> \
  --username backup@pbs --password '<mdp>' --fingerprint '<empreinte>'
vzdump 101 102 110 111 112 113 120 --storage pbs-hp --mode snapshot
```

Résultat : sauvegarde hebdomadaire automatique de toute la nouvelle infra vers l'ancien serveur, avec une restauration réelle testée avant de faire confiance au système.

Tests et Résultats

Test	Commande	Résultat
Comparaison des tables PostgreSQL	<pre>psql -d ncore -c '\dt'</pre> des deux côtés	6 tables identiques
Dépôts Gitea migrés	<pre>curl .../api/v1/repos/search?uid=2</pre>	10 dépôts avec taille non nulle
Services applicatifs up	<pre>docker ps</pre> sur apps	8 services "Up"
HTTP par service	<pre>curl -I http://localhost:<port></pre>	200 sur portfolio, doc-forge, cv-app, intelligence-hub
Snapshot avant modification	<pre>vzdump 101 102 --mode snapshot</pre>	sauvegarde prise avant toute modif risquée
Token Terraform après rotation	<pre>terraform plan</pre>	"No changes" (state cohérent après rotation du secret)
Backup PBS restauré	restauration test d'un vzdump	restauration réelle validée, pas juste supposée

| Difficultés Rencontrées et Solutions

#	Problème	Cause	Solution
1	terraform apply échouait en "permission denied"	Un token Proxmox n'hérite pas des droits de son utilisateur parent	Donner le rôle explicitement au token ET à l'utilisateur (PVEAdmin)
2	Erreur Sys.Modify manquant pour la VM apps	Séparation de privilèges Proxmox : droits effectifs = intersection token/utilisateur	Rôle custom SysModifyOnly ajouté aux deux
3	terraform apply bloqué 13 minutes sur "Still creating"	Terraform attendait le qemu-guest-agent , absent de l'image cloud par défaut	Installation manuelle du qemu-guest-agent , débloque immédiatement
4	Extraction d'archive qui aplatisait tous les projets ensemble	--strip-components=3 au lieu de 2 (mauvais calcul du nombre de niveaux)	Corrigé en --strip-components=2
5	Transfert direct PC → VM apps refusé	Clé SSH non fiée directement vers cette machine	Passage par le bastion draken-admin comme pivot
6	intelligence-hub en crash-loop après redéploiement	Upstream nginx (newsletter-api)	Redémarrage du service dépendant avant le service principal

#	Problème	Cause	Solution
		) resté arrêté après le transfert	

Améliorations Possibles

Sécurité renforcée - Rotation régulière (pas juste ponctuelle) des tokens API
Proxmox et Gitea - Chiffrer les archives de transfert (

```
tar
```

+

```
gpg
```

) plutôt que de les faire transiter en clair entre les machines

Automatisation - Écrire un playbook Ansible qui installe automatiquement

```
qemu-guest-agent
```

sur toute nouvelle VM cloud-init, pour ne plus revivre le blocage de 13 minutes -
Scripter la vérification post-migration (boucle

```
curl
```

sur tous les ports attendus) plutôt que de tester chaque service à la main

Fiabilité - Tester une restauration PBS complète (pas juste un fichier) pour valider le plan de reprise dans son ensemble - Documenter un rollback explicite pour chaque phase de la migration, pas seulement pour la bascule finale

Infrastructure étendue - Isoler complètement l'ancien serveur du réseau de production (ACL Tailscale ou VLAN dédié) une fois son rôle de sauvegarde confirmé stable

Ce que ce Projet Démontre

Compétence	Description
Migration de serveur	Sauvegarde, transfert et redéploiement complet d'une infra Docker vers une nouvelle plateforme, sans perte de données
Diagnostic Proxmox/Terraform	Compréhension de la séparation de privilèges (token vs utilisateur), résolution d'un blocage non documenté (qemu-guest-agent)
Sécurité des transferts	Migration de dépôts Git sans exposer de token, rotation de secrets identifiés comme à risque
Vérification de données	Comparaison de tables PostgreSQL avant/après migration, pas de confiance aveugle dans un dump
Continuité de service	Ancien serveur maintenu intact pendant toute la transition, reconverti ensuite en cible de sauvegarde

Références et Ressources

- Documentation officielle Proxmox VE (permissions, tokens API,

```
vzdump
```

```
)
```

- Documentation du provider Terraform bpg/proxmox
- Documentation officielle PostgreSQL (

```
pg_dumpall
```

```
,
```

```
pg_restore
```

```
)
```

- Documentation officielle Gitea (API admin, adoption de dépôts non gérés)

- Documentation Proxmox Backup Server